PRESENTATION NOTES

# Sections 5-7: decisions, lessons, phase two

Grounded in this repo, not generic teamwork language. Use as talking points. Architecture Decision Records live in docs/Architecture_Decision_Records.docx - cite that they exist; you do not need to read them aloud.

## Section 5 | One significant technical decision

### PostgreSQL over MongoDB (primary example)

*Say: "We chose PostgreSQL, not MongoDB, because a sale is a set of facts that must stay consistent: stock goes down, a sale row is written, and each line gets a cost. That is a transaction, not a document dump."*

Why it fits this shop: products have typed columns (quantity, cost_price, category). Sales stamp unit_price and FIFO unit_cost onto sale_items so later price edits do not rewrite history. Stock layers use FOR UPDATE inside a transaction so two tills cannot oversell the same pack of pens. MongoDB would have been faster to sketch, but we would have re-implemented joins, constraints, and atomic stock updates ourselves. Migrations in database/ are versioned SQL the DB owners can review.

### Web app rather than a native mobile app (supporting decision)

*Say: "We built a React web app, not Android first, because the counter already has a browser, the course timeline was weeks not months, and one codebase could be used on a laptop or a phone browser."*

Offline still matters when ECG or data drops: Dexie queues sales in IndexedDB and POST /api/sync replays them (ADR-005, last-write-wins). A store app would have meant Play Store, Capacitor, and a second UI track we did not have staff time for. Android is explicitly in the later backlog (Won't / phase two).

### Architecture Decision Records

The team keeps Architecture Decision Records in docs/Architecture_Decision_Records.docx. They are also referenced from the root README and the backend/frontend READMEs. You do not need to open the file in the video. Name two that show up in code comments:

ADR-004 - JWT auth and role checks (owner vs assistant) in backend/src/middleware/auth.js. ADR-005 - offline queue and last-write-wins sync (frontend Dexie + POST /api/sync).

### Third-party services (show working, do not invent Paystack)

> **We did not integrate Paystack, Africa's Talking, or Supabase Storage.**
> The shop records sales in-house; there is no card checkout or SMS OTP in this product. Claiming those would be dishonest. What we did integrate is hosting and Postgres as a service.

Neon (Postgres): production data when USE_MOCK_DB=false. On the live site, GET /api/health has returned {"status":"ok","mode":"postgres"} - that means Express is talking to Neon, not the in-memory mock. Show health in a tab, then log in and load Inventory so products persist after refresh.

Vercel (frontend): https://regimens-six.vercel.app - React SPA. Same-origin /api is proxied with API_ORIGIN so the Render URL is not baked in as VITE_.

Render (API): Node Express. Be honest on camera: the original free instance hit monthly hours and could show

'Service Suspended'. KeepAlive pings /api/health every 30s while the tab is open. If health is postgres, the third-party DB path is working. If health fails, say so - do not fake Paystack instead.

## Section 6 | Challenges and lessons learnt (1-2 minutes)

### Hardest technical problem

*Say: "The hardest technical problem was not FIFO math. It was getting a split deploy to stay up: Vercel for the UI, Render for the API, Neon for Postgres."*

What went wrong, specifically: Render's default build ran npm run build at repo root, which called Vite. Vite lives in frontend/node_modules, so the API host failed with 'vite: not found'. We changed root scripts so build/start only prepare and run Express (scripts/render-start.js). Then Vercel warned that VITE_API_URL is a public prefix; we moved the host to API_ORIGIN and a serverless proxy. The first catch-all function only matched one path segment, so /api/health worked and POST /api/auth/login 404ed. We rewrote nested /api/* through frontend/api/proxy.js. Free Render also slept or suspended; the UI then said it could not reach the API even though the React app was fine.

Workaround: Neon for durable data; mock mode only for demos; KeepAlive; document API_ORIGIN; do not point VITE_API_URL at a dead onrender host.

### Hardest non-technical challenge

*Say: "Coordination and hosting quota, not 'the client was difficult'."*

Seven people across docs, DB, backend, UI. Easy to duplicate work or leave Forgot? as a dead span. The client process is a real stationery shop on a notebook - they are not paying for 24/7 Render. When free hours ran out, the 'production' URL died even though the code was correct. Time pressure pushed mock mode so frontend could proceed without waiting for Postgres. Scope: we kept staff roles and receipts as Should/Won't rather than finishing every wish.

### What we would do differently

1. Put Postgres on Neon on day one and treat Render as compute only - one free resource, not web plus Render Postgres. 2. Never bake the API host into VITE_ variables; same-origin /api from the start. 3. Budget a few dollars of always-on API before the marking demo, or accept mock mode on camera and say so. 4. Agree a written MoSCoW with the client early so FIFO and offline are Must, Android is Won't. 5. Run one end-to-end production login in week two, not the night before.

### What we learned that we did not expect

Env-var prefixes (VITE_) are a product decision, not a naming preference - they leak into the browser bundle. Serverless rewrites are not the same as Express routers; a catch-all can 404 nested routes. Free-tier 'production' is an operations problem: sleep, suspend, 60-second cold starts. FIFO is simple on paper and fussy in code: leftover stock must keep its old cost when the shelf price changes.

## Section 7 | What comes next (30s-1 min)

### Phase two

Always-on API, email or owner-issued recovery without a shared shop code, printed/PDF receipts, owner reset of a staff password from Settings, pagination on orders and reports, and a PWA or lightweight Android wrap if the counter wants a home-screen icon.

### Backlog we did not have time to build

Native Android; Paystack or mobile money (not in current sales flow); SMS alerts (Africa's Talking); true pg_dump backups; refresh-token revocation; MFA; multi-branch / store_id; merge (not last-write-wins) for concurrent product edits; rate-limit store in Redis.

### If this had to serve 10,000 users

Stop using one Express process on free Render. Run multiple API instances behind a load balancer, Postgres with a sized connection pool, Redis for rate limits and sessions, paginated APIs, chunked/idempotent sync, and a CDN for the SPA. JWT would need rotation and a revoke list. The schema would need store or tenant ids. That is a rewrite of operations, not a new button on the dashboard.

### Is the client still using it? Support arrangement?

**Be honest on this question.**

Outstretched Arm was the case study. We have a deployed web app and a demo login (owner / owner1234 until they change it). We do not have a paid SLA. Free Render hours ran out on the original API, so 'always on' is not guaranteed. Handover is: Neon connection string, Vercel project, API_ORIGIN, JWT_SECRET, and the GitHub repo. If they keep using it after the course, they (or a teammate) must pay for always-on compute or accept mock/demo resets.